
Cross-Browser Test Result Visualizer

Technical Project Report
CSE3253 — DevOps Engineering

Student Name: Ananya Jain
Course Code: CSE3253
Project Type: Jenkins & CI + Selenium Automation
Difficulty Level: Intermediate
Submission Date: March 24, 2026

This report documents the design, implementation, and deployment of a centralized cross-browser Selenium test result visualization dashboard integrated with a Jenkins CI/CD pipeline.

Abstract

In modern DevOps environments, automated testing across multiple browsers is a fundamental quality assurance practice. However, one of the persistent challenges faced by engineering teams is the fragmentation of test output — results are distributed across terminal logs, JUnit XML files, and browser-specific reports, making holistic analysis difficult and time-consuming.

This project presents the **Cross-Browser Test Result Visualizer**, a full-stack web application that consolidates Selenium-based automated test results from multiple browsers (Chrome, Firefox, and Edge) into a single, interactive dashboard. The system provides real-time comparison of pass/fail ratios, execution time analysis, and historical trend monitoring.

The solution is built on a Python/Flask REST API backend, a PostgreSQL database for persistent storage, a Chart.js-powered frontend dashboard, and is fully containerized with Docker and Docker Compose. Continuous integration and delivery are orchestrated by a Jenkins CI pipeline defined through a **Jenkinsfile**, enabling automated test execution and deployment on every code push.

The result is a production-grade DevOps tool that bridges the gap between raw test execution and actionable quality metrics, demonstrating practical integration of software testing, containerization, and CI/CD automation.

Keywords: Selenium, Cross-Browser Testing, Jenkins, CI/CD, Flask, Docker, PostgreSQL, DevOps, Chart.js, Pytest

Contents

1. Introduction

1.1 Background and Motivation

Software quality assurance in the DevOps era demands fast, automated, and reliable testing pipelines. Cross-browser compatibility is a non-trivial concern: web applications may behave differently across Chrome, Firefox, and Edge due to differences in JavaScript engines, CSS rendering engines, and browser-specific API implementations.

Selenium WebDriver is the industry-standard framework for browser automation and cross-browser testing. However, in typical CI/CD pipelines, Selenium test results are consumed once — logged to console or written to XML — and rarely aggregated or visualized in a meaningful way.

This project was conceived to address a real-world pain point: *how can a team quickly understand cross-browser test health, identify regressions, and monitor execution trends over time?*

1.2 Problem Statement

In real-world DevOps environments, Selenium tests are executed across multiple browsers. The resulting data is typically scattered across:

- Terminal log output (non-persistent, ephemeral)
- JUnit-format XML files generated by Pytest
- Browser-specific CI job artifacts
- Manual test reports maintained by engineers

This fragmentation makes it difficult to:

- Compare browser-by-browser pass/fail performance
- Identify intermittent failures or flaky tests quickly
- Visualize execution time trends across builds
- Monitor overall CI test quality over time

1.3 Project Objectives

This project was designed to achieve the following concrete objectives:

1. **Automatic Collection:** Automatically collect Selenium test results via a REST API endpoint at the conclusion of each test run.
2. **Persistent Storage:** Store all test result records in a PostgreSQL relational database with structured schema.
3. **Interactive Visualization:** Render an interactive web dashboard using Chart.js that displays pass/fail comparisons and execution time metrics.
4. **CI Integration:** Integrate the system with a Jenkins pipeline that executes tests and updates the dashboard on every build.
5. **Containerization:** Package all components (Flask app, PostgreSQL) using Docker and Docker Compose for reproducible, environment-agnostic deployment.

1.4 Scope

The project covers the end-to-end lifecycle of a cross-browser test result: from Selenium script execution, through API ingestion, database persistence, and finally visual presentation. It is scoped to Chrome and Firefox browser automation in the primary implementation, with Edge supported by the architecture. The project explicitly addresses DevOps concerns: containerization, CI/CD pipeline automation, and infrastructure-as-code.

1.5 Report Structure

The remainder of this report is organized as follows:

- **Chapter 2** covers the system architecture and design decisions.
- **Chapter 3** describes the technology stack in detail.
- **Chapter 4** details the implementation of each component.
- **Chapter 5** explains the CI/CD pipeline design.
- **Chapter 6** covers containerization and Docker setup.
- **Chapter 7** documents the REST API specification.
- **Chapter 8** describes the frontend dashboard.
- **Chapter 9** covers testing strategy and execution.
- **Chapter 10** presents conclusions and future work.

2. System Architecture

2.1 Architectural Overview

The system follows a classic three-tier web application architecture augmented with a dedicated test execution layer and a CI/CD automation layer. The five logical layers are:

1. **Test Execution Layer** — Selenium WebDriver scripts execute automated tests in browser instances.
2. **API Layer** — Flask REST API receives test results and serves dashboard data.
3. **Persistence Layer** — PostgreSQL database stores all test result records.
4. **Presentation Layer** — HTML/JavaScript dashboard renders charts and statistics.
5. **CI/CD Layer** — Jenkins pipeline orchestrates execution, collection, and deployment.

2.2 Data Flow

The primary data flow through the system is as follows:

Test Execution Pipeline:

Selenium Test Script → HTTP POST `/api/results` → Flask API → SQLAlchemy ORM → PostgreSQL → Dashboard Query → Chart.js Visualization

Each Selenium test script — whether executing in Chrome or Firefox — follows an identical reporting protocol: upon completion (pass or fail), the script constructs a JSON payload containing the browser name, test status, and execution duration, and sends it via HTTP POST to the Flask API endpoint.

2.3 Component Interaction Diagram

The interaction between the major system components is described in the following table:

Source Component	Target Component	Interaction Description
Selenium (Chrome)	Flask API	HTTP POST of test result JSON
Selenium (Firefox)	Flask API	HTTP POST of test result JSON
Flask API	PostgreSQL	INSERT via SQLAlchemy ORM
Browser (Dashboard)	Flask API	HTTP GET /api/results
Flask API	Browser	JSON array of all test results
Jenkins Pipeline	Docker Host	docker-compose up / pytest run

Table 2.1: Component interaction summary

2.4 Project Directory Structure

The repository is organized following separation-of-concerns principles, with source code, tests, infrastructure, and pipeline definitions in distinct directories:

```

1 devopsprojectcrossbrowservisualizer/
2 |
3 |-- README.md
4 |-- requirements.txt
5 |-- docker-compose.yml
6 |
7 |-- src/
8 |   |
9 |   |   |-- main/
10 |   |   |   |-- app.py           # Flask application entry point
11 |   |   |   |-- models.py        # SQLAlchemy database models
12 |   |   |   |-- routes.py        # API endpoints and dashboard routes
13 |   |   |   |-- config.py        # Application configuration
14 |   |   |   |-- extensions.py     # Flask extension initialization
15 |   |   |   |-- templates/
16 |   |   |       |-- dashboard.html # Jinja2 dashboard template
17 |   |-- tests/
18 |   |   |-- selenium/
19 |   |   |   |-- chrome.py         # Chrome test suite
20 |   |   |   |-- firefox.py       # Firefox test suite
21 |   |-- infrastructure/
22 |   |   |-- docker/
23 |   |   |   |-- Dockerfile        # Application container image
24 |   |   |   |-- wait-for-db.sh    # DB readiness health check
25 |   |   |-- script
26 |   |-- pipelines/
27 |   |   |-- Jenkinsfile           # Declarative Jenkins pipeline
28 |

```

Listing 2.1: Repository Directory Structure

3. Technology Stack

3.1 Overview

The project is built on a carefully chosen set of modern, open-source technologies. The selection prioritizes production relevance, DevOps toolchain compatibility, and community support.

3.2 Core Technologies

3.2.1 Python 3.10+

Python serves as the primary programming language for both the backend application and the test automation scripts. Python 3.10 or later is required to leverage modern language features including structural pattern matching and improved type annotation support. The language's rich ecosystem of web and testing libraries makes it ideal for this use case.

3.2.2 Flask

Flask is a lightweight WSGI web framework that provides the REST API backbone and server-side rendering capability for the dashboard. Its microframework philosophy avoids unnecessary abstractions, making the routing logic transparent and testable. Flask's built-in Jinja2 template engine is used to render the `dashboard.html` view.

3.2.3 SQLAlchemy

SQLAlchemy provides the Object-Relational Mapping (ORM) layer between the Flask application and PostgreSQL. It abstracts raw SQL into Python class definitions, enabling clean model definitions and type-safe database operations. The `models.py` file defines the `TestResult` model that maps to the database schema.

3.2.4 PostgreSQL

PostgreSQL is used as the primary relational database for persistent test result storage. It was chosen over lighter alternatives (SQLite, MySQL) for its robustness, support for concurrent connections from multiple test runners, and compatibility with production-grade Docker deployments.

3.2.5 Selenium WebDriver

Selenium WebDriver is the industry-standard library for programmatic browser control. Test scripts in `tests/selenium/chrome.py` and `tests/selenium/firefox.py` use the respective WebDriver instances to launch browser sessions, execute test scenarios, capture pass/fail status, and measure execution duration.

3.2.6 Chart.js

Chart.js is a JavaScript charting library used to render the frontend visualizations. It provides animated, responsive bar charts, doughnut charts, and line charts that display browser comparison metrics and execution time distributions directly in the dashboard.

3.3 DevOps Tools

3.3.1 Git

Git is used for version control throughout the project. The repository structure and commit history document the incremental development of the application, infrastructure code, and pipeline definitions.

3.3.2 Docker

Docker is used to containerize the Flask application. The `Dockerfile` in `infrastructure/docker/` defines the application image build process, including Python dependency installation and application startup configuration.

3.3.3 Docker Compose

Docker Compose orchestrates the multi-container deployment, managing both the Flask application container and the PostgreSQL database container as a cohesive service group. The `docker-compose.yml` at the project root defines service configurations, port mappings, environment variables, and inter-service dependencies.

3.3.4 Jenkins

Jenkins is the CI/CD automation server. The declarative pipeline defined in `pipelines/Jenkinsfile` automates the sequence of: cloning the repository, building Docker images, running Selenium test suites, and deploying the updated application.

3.3.5 Pytest

Pytest is the test runner used to execute the Selenium test scripts. It provides structured test discovery, fixture management, and integration with JUnit XML reporting for Jenkins consumption.

3.4 Technology Summary Table

Component	Technology	Version	Role
Backend Language	Python	3.10+	Application & Test Logic
Web Framework	Flask	2.x	REST API & Routing
ORM	SQLAlchemy	2.x	Database Abstraction
Database	PostgreSQL	14+	Persistent Storage
Browser Automation	Selenium WebDriver	4.x	Cross-Browser Testing
Frontend Charting	Chart.js	4.x	Data Visualization
Test Runner	Pytest	7.x	Test Execution
Containerization	Docker	24+	App Packaging
Orchestration	Docker Compose	v2	Multi-Container Mgmt
CI/CD Server	Jenkins	LTS	Pipeline Automation
Version Control	Git	2.x	Source Management

Table 3.1: Complete technology stack summary

4. Implementation

4.1 Backend Application

4.1.1 Application Entry Point (app.py)

The Flask application is initialized in `src/main/app.py`. This file is responsible for creating the Flask app instance, registering configuration, initializing extensions (database, migrations), and registering the route blueprints.

```
1 from flask import Flask
2 from extensions import db
3 from routes import main_bp
4 from config import Config
5
6 def create_app():
7     app = Flask(__name__)
8     app.config.from_object(Config)
9
10    db.init_app(app)
11
12    with app.app_context():
13        db.create_all()
14
15    app.register_blueprint(main_bp)
16    return app
17
18 if __name__ == '__main__':
19     app = create_app()
20     app.run(host='0.0.0.0', port=8080)
```

Listing 4.1: Typical Flask Application Factory Pattern

4.1.2 Database Model (models.py)

The `TestResult` model defines the schema for storing individual test execution records. Each record captures the browser under test, the outcome status, and the test duration in seconds.

```
1 from extensions import db
2 from datetime import datetime
3
4 class TestResult(db.Model):
5     __tablename__ = 'test_results'
6
```

```
7     id      = db.Column(db.Integer, primary_key=True)
8     browser  = db.Column(db.String(50), nullable=False)
9     status   = db.Column(db.String(20), nullable=False)
10    duration  = db.Column(db.Float, nullable=False)
11    timestamp = db.Column(db.DateTime, default=datetime.utcnow)
12
13    def to_dict(self):
14        return {
15            'id':      self.id,
16            'browser': self.browser,
17            'status':  self.status,
18            'duration': self.duration,
19            'timestamp': self.timestamp.isoformat(),
20        }
```

Listing 4.2: TestResult SQLAlchemy Model

4.1.3 API Routes (routes.py)

The routes module defines two primary REST endpoints: one to submit a new test result, and one to retrieve all historical results for dashboard rendering.

```
1 from flask import Blueprint, request, jsonify, render_template
2 from models import TestResult
3 from extensions import db
4
5 main_bp = Blueprint('main', __name__)
6
7 @main_bp.route('/api/results', methods=['POST'])
8 def submit_result():
9     data = request.get_json()
10    result = TestResult(
11        browser = data['browser'],
12        status  = data['status'],
13        duration = data['duration']
14    )
15    db.session.add(result)
16    db.session.commit()
17    return jsonify({'success': True}), 201
18
19 @main_bp.route('/api/results', methods=['GET'])
20 def get_results():
21     results = TestResult.query.order_by(
22         TestResult.timestamp.desc()
23     ).all()
24     return jsonify([r.to_dict() for r in results])
25
26 @main_bp.route('/')
27 def dashboard():
28     return render_template('dashboard.html')
```

Listing 4.3: Flask Route Definitions

4.2 Selenium Test Scripts

4.2.1 Chrome Test Script (tests/selenium/chrome.py)

The Chrome test script initializes a headless Chrome WebDriver, executes the defined test scenarios, records the outcome and elapsed time, and POSTs the result to the Flask API.

```
1 import time, requests
2 from selenium import webdriver
3 from selenium.webdriver.chrome.options import Options
4
5 API_URL = "http://localhost:8080/api/results"
6
7 def test_home_page_chrome():
8     options = Options()
9     options.add_argument("--headless")
10    driver = webdriver.Chrome(options=options)
11
12    start = time.time()
13    status = "pass"
14
15    try:
16        driver.get("http://localhost:8080")
17        assert "Dashboard" in driver.title
18    except Exception:
19        status = "fail"
20    finally:
21        duration = round(time.time() - start, 3)
22        driver.quit()
23
24    requests.post(API_URL, json={
25        "browser": "chrome",
26        "status": status,
27        "duration": duration,
28    })
```

Listing 4.4: Chrome Selenium Test with Result Reporting

4.2.2 Firefox Test Script (tests/selenium/firefox.py)

The Firefox test script mirrors the Chrome implementation but uses `GeckoDriver` and `FirefoxOptions`. This parallel structure ensures consistent result schema regardless of browser.

4.2.3 Test Execution Commands

Tests are executed via Pytest with the following commands:

```
1 # Activate virtual environment
2 python3 -m venv venv
3 source venv/bin/activate
4 pip install -r requirements.txt
5
6 # Run Chrome tests
7 pytest tests/selenium/chrome.py -v
8
9 # Run Firefox tests
```

```
10 pytest tests/selenium/firefox.py -v
11
12 # Run all tests and generate JUnit XML report
13 pytest tests/ --junitxml=test-results.xml
```

Listing 4.5: Running Tests with Pytest

5. CI/CD Pipeline

5.1 Jenkins Overview

Jenkins is a widely adopted open-source automation server that enables building, testing, and deploying software through configurable pipelines. In this project, Jenkins orchestrates the full DevOps workflow from code commit to running dashboard update.

5.2 Pipeline Design

The pipeline is defined declaratively in `pipelines/Jenkinsfile`. A declarative pipeline enforces a clear, readable structure and integrates naturally with Jenkins Blue Ocean and GitHub webhooks for automatic triggering.

The pipeline is divided into the following stages:

Stage #	Stage Name	Description
1	Checkout	Clone repository from source control
2	Build	Build Docker images via docker-compose
3	Deploy	Start all containers in detached mode
4	Test	Execute Chrome and Firefox Selenium suites
5	Publish	Archive JUnit test results in Jenkins
6	Cleanup	Stop and remove containers post-run

Table 5.1: Jenkins pipeline stages

5.3 Jenkinsfile

```
1 pipeline {
2     agent any
3
4     environment {
5         COMPOSE_FILE = 'docker-compose.yml'
6     }
```

```
7
8   stages {
9       stage('Checkout') {
10           steps {
11               checkout scm
12           }
13       }
14
15       stage('Build') {
16           steps {
17               sh 'docker-compose build'
18           }
19       }
20
21       stage('Deploy') {
22           steps {
23               sh 'docker-compose up -d'
24               sh 'sleep 10' // Allow DB and app to initialize
25           }
26       }
27
28       stage('Test - Chrome') {
29           steps {
30               sh '''
31                   python3 -m venv venv
32                   . venv/bin/activate
33                   pip install -r requirements.txt
34                   pytest tests/selenium/chrome.py \
35                       --junitxml=results-chrome.xml -v
36               '''
37           }
38       }
39
40       stage('Test - Firefox') {
41           steps {
42               sh '''
43                   . venv/bin/activate
44                   pytest tests/selenium/firefox.py \
45                       --junitxml=results-firefox.xml -v
46               '''
47           }
48       }
49
50       stage('Publish Results') {
51           steps {
52               junit 'results-*.xml'
53           }
54       }
55   }
56
57   post {
58       always {
59           sh 'docker-compose down'
60       }
61       success {
62           echo 'Pipeline completed successfully.'
63       }
64       failure {
```

```
65         echo 'Pipeline failed. Review test results.'
66     }
67 }
68 }
```

Listing 5.1: Declarative Jenkinsfile

5.4 Trigger Strategy

The pipeline can be configured in Jenkins to trigger on:

- **Push events** via GitHub/GitLab webhooks (immediate trigger on every commit)
- **Scheduled execution** via cron syntax (e.g., nightly regression runs)
- **Manual trigger** from the Jenkins dashboard for on-demand testing

6. Containerization with Docker

6.1 Docker Architecture

The application uses a multi-container Docker architecture managed by Docker Compose. Two services are defined: the Flask application service and the PostgreSQL database service. A shared Docker network enables inter-container communication using service names as DNS hostnames.

6.2 Dockerfile

The Dockerfile in `infrastructure/docker/` defines the application container build:

```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 # Install system dependencies
6 RUN apt-get update && apt-get install -y \
7     gcc \
8     libpq-dev \
9     && rm -rf /var/lib/apt/lists/*
10
11 # Install Python dependencies
12 COPY requirements.txt .
13 RUN pip install --no-cache-dir -r requirements.txt
14
15 # Copy application source
16 COPY src/main/ .
17
18 # Copy DB readiness script
19 COPY infrastructure/docker/wait-for-db.sh /wait-for-db.sh
20 RUN chmod +x /wait-for-db.sh
21
22 EXPOSE 8080
23
24 CMD ["/wait-for-db.sh", "db", "python", "app.py"]
```

Listing 6.1: Application Dockerfile

6.3 Docker Compose Configuration

```
1 version: '3.8'
2
3 services:
4   app:
5     build:
6       context: .
7       dockerfile: infrastructure/docker/Dockerfile
8     ports:
9       - "8080:8080"
10    environment:
11      - DATABASE_URL=postgresql://user:pass@db:5432/testresults
12    depends_on:
13      - db
14    networks:
15      - app-network
16
17    db:
18      image: postgres:14-alpine
19      environment:
20        POSTGRES_USER: user
21        POSTGRES_PASSWORD: pass
22        POSTGRES_DB: testresults
23      volumes:
24        - pgdata:/var/lib/postgresql/data
25      networks:
26        - app-network
27
28    volumes:
29      pgdata:
30
31    networks:
32      app-network:
33        driver: bridge
```

Listing 6.2: docker-compose.yml

6.4 Database Readiness Script

The `wait-for-db.sh` script prevents the Flask application from starting before PostgreSQL is ready to accept connections — a common race condition in Docker Compose deployments.

```
1 #!/bin/sh
2 # wait-for-db.sh: Wait for PostgreSQL to be ready
3
4 HOST=$1
5 shift
6 CMD="$@"
7
8 until pg_isready -h "$HOST" -U "user"; do
9   echo "Waiting for database at $HOST..."
10   sleep 2
11 done
12
13 echo "Database is ready. Starting application..."
14 exec $CMD
```

Listing 6.3: wait-for-db.sh health check script

6.5 Running the Application

```
1 # Build and start all containers in detached mode
2 docker-compose up --build -d
3
4 # View running containers
5 docker-compose ps
6
7 # View application logs
8 docker-compose logs -f app
9
10 # Stop and remove containers
11 docker-compose down
12
13 # Stop containers and remove volumes (full reset)
14 docker-compose down -v
```

Listing 6.4: Docker Compose deployment commands

Once running, the dashboard is accessible at <http://localhost:8080>.

7. REST API Documentation

7.1 API Overview

The Flask backend exposes a simple RESTful API with three endpoints. All request and response bodies use JSON encoding. The API serves both as the data ingestion point for Selenium test scripts and as the data source for the frontend dashboard.

7.2 Base URL

```
http://localhost:8080
```

7.3 Endpoints

7.3.1 Submit Test Result

Method	POST
Path	/api/results
Content-Type	application/json

Request Body:

```
1 {
2   "browser": "chrome",
3   "status": "pass",
4   "duration": 2.5
5 }
```

Listing 7.1: POST /api/results request body

Field Descriptions:

Response (201 Created):

```
1 {
2   "success": true
3 }
```

Listing 7.2: POST /api/results response

Field	Type	Required	Description
browser	string	Yes	Browser name: chrome, firefox, edge
status	string	Yes	Test outcome: pass or fail
duration	float	Yes	Execution time in seconds

Table 7.1: POST /api/results field definitions

7.3.2 Retrieve All Test Results

Method	GET
Path	/api/results
Content-Type	application/json

Response (200 OK):

```
1  [
2    {
3      "id":      1,
4      "browser": "chrome",
5      "status":  "pass",
6      "duration": 2.5,
7      "timestamp": "2024-11-15T10:23:45.123456"
8    },
9    {
10     "id":      2,
11     "browser": "firefox",
12     "status":  "fail",
13     "duration": 3.8,
14     "timestamp": "2024-11-15T10:25:10.654321"
15   }
16 ]
```

Listing 7.3: GET /api/results response

7.3.3 Dashboard View

Method	GET
Path	/
Returns	text/html
Description	Renders the interactive test result dashboard

8. Frontend Dashboard

8.1 Dashboard Overview

The dashboard is a single-page web application served by Flask at the root URL (/). It provides an at-a-glance view of cross-browser test health, including pass/fail breakdowns by browser, execution time comparisons, and historical test volume.

8.2 Visualizations

The dashboard includes the following Chart.js-powered visualizations:

Chart Type	Chart.js Type	Purpose
Browser Pass/Fail	Grouped Bar Chart	Compare pass and fail counts per browser side-by-side
Pass Rate Distribution	Doughnut Chart	Show overall pass rate as a percentage
Execution Time	Bar Chart	Compare average test duration per browser
Test Volume Over Time	Line Chart	Track cumulative test runs over time

Table 8.1: Dashboard chart components

8.3 Dashboard Data Flow

On page load, the dashboard JavaScript fetches all test results from `GET /api/results` and processes the JSON array client-side to compute:

- Per-browser pass count and fail count
- Per-browser average execution duration
- Overall pass rate percentage
- Time-series test volume for trend analysis

The computed metrics are then passed to Chart.js dataset configurations to render the visualizations.

8.4 Dashboard HTML Structure

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <title>Test Result Dashboard</title>
6   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
7 </head>
8 <body>
9   <h1>Cross-Browser Test Results</h1>
10
11   <!-- Summary cards -->
12   <div id="summary-cards"></div>
13
14   <!-- Charts -->
15   <canvas id="passFailChart"></canvas>
16   <canvas id="durationChart"></canvas>
17   <canvas id="trendChart"></canvas>
18
19   <script>
20     fetch('/api/results')
21       .then(res => res.json())
22       .then(data => renderDashboard(data));
23
24     function renderDashboard(results) {
25       // Process results and render Chart.js charts
26     }
27   </script>
28 </body>
29 </html>
```

Listing 8.1: Dashboard HTML skeleton (dashboard.html)

9. Testing Strategy

9.1 Testing Layers

The project employs a two-layer testing strategy:

1. **Selenium End-to-End Tests:** Browser automation tests that simulate user interactions with the target web application and validate its behavior.
2. **API Integration Tests:** Pytest tests that directly call the Flask API endpoints to validate request handling, database writes, and response formats.

9.2 Selenium Test Scope

The Selenium test scripts cover the following scenarios:

Test Case	Browser(s)	Expected Result	Metric Captured
Homepage Load	Chrome, Firefox	Page loads, title matches	Duration (ms)
Dashboard Chart Render	Chrome, Firefox	Canvas elements present	Duration (ms)
API POST Response	Chrome, Firefox	HTTP 201 received	Duration (ms)
Results Table Population	Chrome, Firefox	Table rows rendered	Duration (ms)

Table 9.1: Selenium test cases

9.3 Test Reporting

Pytest generates JUnit-format XML reports (`results-chrome.xml`, `results-firefox.xml`) which are consumed by Jenkins using the `junit` post-build step. This enables:

- Per-build test trend graphs in the Jenkins UI
- Email/Slack notifications on test failures
- Historical test pass rate tracking

9.4 Headless Browser Configuration

All Selenium tests run in headless mode to support execution in CI environments (Jenkins agents, Docker containers) that do not have a display server. Headless Chrome and Firefox are configured via their respective options classes:

```
1 # Chrome headless
2 from selenium.webdriver.chrome.options import Options as
   ChromeOptions
3 chrome_opts = ChromeOptions()
4 chrome_opts.add_argument("--headless")
5 chrome_opts.add_argument("--no-sandbox")
6 chrome_opts.add_argument("--disable-dev-shm-usage")
7
8 # Firefox headless
9 from selenium.webdriver.firefox.options import Options as
   FirefoxOptions
10 ff_opts = FirefoxOptions()
11 ff_opts.add_argument("--headless")
```

Listing 9.1: Headless browser options configuration

10. Conclusion and Future Work

10.1 Summary of Achievements

This project successfully delivered a complete, production-oriented DevOps solution for cross-browser test result visualization. The following objectives were fully achieved:

- **Automated test result collection** via REST API POST endpoint, eliminating manual result aggregation.
- **Persistent storage** in PostgreSQL with a structured, timestamped schema for historical analysis.
- **Interactive visualization dashboard** using Chart.js, providing instant insight into browser-by-browser pass/fail metrics and execution times.
- **Full containerization** with Docker and Docker Compose, enabling one-command deployment on any Docker-capable host.
- **Jenkins CI/CD pipeline** with declarative Jenkinsfile, automating the full cycle from commit to tested deployment.
- **Headless Selenium testing** for Chrome and Firefox, compatible with CI execution environments.

10.2 Challenges Encountered

10.2.1 Docker Service Startup Race Condition

PostgreSQL takes several seconds to initialize after container start, causing the Flask app to fail on database connection if started immediately. This was resolved by the `wait-for-db.sh` script, which polls `pg_isready` before launching the application process.

10.2.2 Headless Browser Driver Compatibility

Maintaining compatibility between Selenium, ChromeDriver, and the installed Chrome binary version requires careful version pinning in `requirements.txt` and the Dockerfile. Selenium Manager (introduced in Selenium 4.6) partially automates this.

10.2.3 Network Isolation in Docker

Selenium test scripts running outside Docker (on the Jenkins agent) need to reach the Flask app running inside Docker. This requires correct port exposure (8080:8080) and using the host machine's IP rather than `localhost` from within a container context.

10.3 Future Enhancements

The following enhancements would extend the project towards a production-ready internal tool:

1. **Edge Browser Support:** Add `tests/selenium/edge.py` using Microsoft Edge WebDriver to complete the major browser trifecta.
2. **Authentication:** Implement JWT-based API authentication to prevent unauthorized result submission.
3. **Test Suite Granularity:** Extend the data model to capture individual test case names and failure messages, not just aggregate pass/fail status.
4. **Alerting Integration:** Add Slack or email webhook notifications when a build's pass rate falls below a configurable threshold.
5. **Historical Trend Analytics:** Implement time-series trend charts showing pass rate evolution across multiple Jenkins builds.
6. **Kubernetes Deployment:** Migrate from Docker Compose to a Kubernetes Helm chart for production-scale orchestration.
7. **Parallel Test Execution:** Use Pytest-xdist or Selenium Grid to run Chrome and Firefox tests in parallel, reducing total pipeline duration.

10.4 Learning Outcomes

This project provided hands-on experience with the complete DevOps toolchain: from writing browser automation scripts, designing REST APIs, managing relational databases, containerizing applications, to configuring CI/CD pipelines. The integration challenges encountered and resolved — particularly around Docker networking, browser driver management, and pipeline sequencing — reflect real-world DevOps engineering scenarios.

References

- [1] Selenium Project. (2024). *Selenium WebDriver Documentation*. <https://www.selenium.dev/documentation/>
- [2] Flask Project. (2024). *Flask Documentation*. <https://flask.palletsprojects.com/>
- [3] SQLAlchemy. (2024). *SQLAlchemy ORM Documentation*. <https://docs.sqlalchemy.org/>
- [4] PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. <https://www.postgresql.org/docs/>
- [5] Docker Inc. (2024). *Docker Documentation*. <https://docs.docker.com/>
- [6] Jenkins Project. (2024). *Jenkins Pipeline Documentation*. <https://www.jenkins.io/doc/book/pipeline/>
- [7] Chart.js Contributors. (2024). *Chart.js Documentation*. <https://www.chartjs.org/docs/>
- [8] Pytest Team. (2024). *Pytest Documentation*. <https://docs.pytest.org/>